

UNIVERSIDADE DE MOGI DAS CRUZES

**CURSO SUPERIOR DE TECNOLOGIA DE ANÁLISE E DESENVOLVIMENTO DE
SISTEMAS – PROJETO DE SOFTWARE E SEGURANÇA DA INFORMAÇÃO**

ALEF DOS SANTOS – 11251104979

ARTHUR CASELLI – 11251104886

RICHARD ARMOND – 11251105106

VITOR SANTANA – 11251105195

PROFESSORES: GEAN ROCHA E WILLIAN SAGARIO

CRIAÇÃO DE SISTEMA PARA GERENCIAMENTO DE ADEGA

Mogi das Cruzes, SP 2026

ALEF DOS SANTOS – 11251104979

ARTHUR CASELLI – 11251104886

RICHARD ARMOND – 11251105106

VITOR SANTANA – 11251105195

CRIAÇÃO DE SISTEMA PARA GERENCIAMENTO DE ADEGA

Trabalho apresentado como avaliação final do semestre no curso de Análise e Desenvolvimento de Sistemas da Universidade de Mogi das Cruzes (UMC). Documentação do sistema de gerenciamento de adega.

Análise e Desenvolvimento de Sistemas – Projeto de Software e Segurança da Informação.

Nome dos orientadores: Gean Rocha e Willian Sagario

Mogi das Cruzes, SP 2026

SUMÁRIO

1	INTRODUÇÃO	3
2	OBJETIVO	4
	2.1 Objetivo Geral	4
	2.2 Objetivos Específicos	4
3	FUNDAMENTAÇÃO TEÓRICA	5
4	DESENVOLVIMENTO DO PROJETO	7
	4.1 Modelagem de dados	7
	4.2 Front-end	10
	4.3 Back-end	10
	4.4 Banco de dados	11
	4.5 Segurança da Informação	12
5	TESTES E RESULTADOS	14
6	CONCLUSÃO	27
7	REFERÊNCIAS	28

1 INTRODUÇÃO

Hoje em dia, usar a tecnologia a favor do negócio não é mais um luxo ou um diferencial, mas sim uma questão de sobrevivência para as pequenas empresas. Estabelecimentos menores, como adegas e distribuidoras de bebidas, lidam com uma correria diária e desafios complexos: o estoque que muda a toda hora, vendas sendo feitas ao mesmo tempo no dinheiro, PIX e cartão, e a necessidade de saber exatamente como está o caixa no fim do dia. Quando a loja não tem uma ferramenta automatizada para ajudar, é muito comum acontecerem erros humanos, como furos no estoque, valores de caixa que não batem e até a falta de segurança na hora de guardar as informações do negócio.

Foi para resolver esse problema que desenvolvemos o Adegos Systems, um sistema de gestão criado exatamente para facilitar, automatizar e proteger o dia a dia desses pequenos comércios. A grande diferença do nosso projeto é que não criamos apenas mais um programa de vendas comum. Nós construímos o sistema pensando em segurança desde o primeiro passo (um conceito conhecido como Security by Design). Na prática, isso significa que juntamos as boas práticas de desenvolvimento de software com regras rigorosas de proteção, evitando que pessoas não autorizadas acessem os dados ou que o sistema fique vulnerável a ataques.

Para que tudo funcione de forma rápida e organizada, dividimos a estrutura do projeto. O "motor" do sistema (o back-end) foi feito em Java usando o Spring Boot, o que garante um ótimo desempenho e deixa as regras do negócio bem protegidas. Já a tela que o usuário vê (o front-end) foi montada com Thymeleaf e JavaScript, criando uma interface rápida e agradável de usar, enquanto todas as informações ficam salvas com segurança em um banco de dados MySQL. Nas próximas páginas deste documento, vamos mostrar o passo a passo de como construímos o sistema, desde o desenho da arquitetura e as escolhas de tecnologia, até as regras de segurança e os testes que colocamos em prática.

2 OBJETIVO

2.1 Objetivo Geral

O objetivo principal deste projeto é desenvolver um sistema de gestão em Java com banco de dados MySQL voltado para pequenos negócios. A ideia é automatizar as rotinas de vendas, o controle de estoque e o gerenciamento dos funcionários, garantindo sempre que as informações da empresa estejam seguras, corretas e protegidas contra acessos indevidos.

2.2 Objetivos Específicos

Para chegar nesse resultado, dividimos o projeto em metas mais diretas. Na parte de segurança, o primeiro passo é proteger as senhas dos usuários usando criptografia (hashing), para que ninguém consiga ler esses dados diretamente no banco de dados. Além disso, vamos criar perfis de acesso diferentes (o chamado controle RBAC). Com isso, o sistema separa certinho o que um Vendedor pode acessar no dia a dia das funções mais críticas (como apagar uma venda), que ficam liberadas apenas para os Administradores.

Pensando no funcionamento prático da loja, a meta é criar uma baixa automática de estoque. Ou seja, toda vez que uma venda for fechada ou uma retirada for registrada, o sistema já atualiza a quantidade do inventário em tempo real. Também vamos montar um módulo de caixa flexível, onde o cliente possa pagar a mesma compra usando dinheiro, PIX e cartão ao mesmo tempo, com o próprio sistema fazendo as contas para garantir que o valor total bate certinho antes de fechar a venda.

Por fim, para garantir que a loja não perca o histórico de nada, aplicaremos a regra de "exclusão lógica" (soft delete). Isso significa que, se alguém apagar um produto ou uma venda, o sistema apenas oculta essa informação em vez de deletar de verdade, deixando tudo registrado para auditoria. Junto a isso, vamos entregar um painel inicial (dashboard) com gráficos visuais e fáceis de entender, mostrando o resumo das vendas diárias, semanais e mensais para ajudar o dono do negócio a tomar as melhores decisões.

3 FUNDAMENTAÇÃO TEÓRICA

Para construir o Adego Systems, escolhemos tecnologias muito usadas no mercado atual, focando em ferramentas que fossem rápidas, seguras e que permitissem o sistema crescer no futuro sem problemas. O "motor" do nosso projeto foi feito em Java (na versão 21). O Java é ótimo porque organiza muito bem o código e funciona em praticamente qualquer computador. Para não termos que configurar tudo do zero, adotamos o Spring Boot (versão 3.4.2). Ele acelera bastante o nosso trabalho, pois já traz um servidor embutido (o Tomcat) e conecta as peças do sistema automaticamente. Na hora de conversar com o banco de dados, usamos o JPA junto com o Hibernate. Basicamente, em vez de ficarmos digitando códigos SQL longos e complexos na mão, o Hibernate "traduz" as nossas classes do Java direto para as tabelas do banco. Isso faz o trabalho pesado por nós e garante que ações combinadas (como apagar uma venda e os itens dela ao mesmo tempo) aconteçam de forma perfeita.

Para guardar as informações da loja de verdade, escolhemos o banco de dados MySQL. Ele é muito confiável porque segue regras rígidas de segurança (conhecidas como regras ACID). Na prática, isso significa que ele não deixa uma venda ser salva pela metade se acabar a energia ou a internet cair, garantindo que os dados do caixa estejam sempre corretos. Já na parte de segurança, que é o grande foco do projeto, usamos o Spring Security junto com tokens JWT. Em sistemas antigos, o servidor precisava ficar "lembrando" quem estava logado, o que gastava muita memória. Nós mudamos isso: quando o funcionário faz login, ele recebe um "crachá digital" secreto e assinado (o JWT). Toda vez que ele clica em algo, o sistema olha para esse crachá para ter certeza de que é ele mesmo, impedindo que invasores falsifiquem ações. E para que ninguém nunca descubra as senhas dos usuários, usamos a tecnologia BCrypt, que embaralha a senha original de um jeito tão complexo que nem mesmo o dono do sistema consegue reverter.

Por fim, na tela em que o usuário realmente clica e trabalha, usamos o Thymeleaf trabalhando em equipe com as ferramentas clássicas da web: HTML5, CSS3 e JavaScript. Essa combinação nos deixou criar um visual moderno, com cores bem contrastantes que não cansam a vista durante o expediente, e que se adapta sozinho tanto na tela do computador quanto no celular do gerente. Para que o sistema não fique recarregando a página e travando o caixa toda hora, usamos uma tecnologia

do JavaScript chamada Fetch API. Ela faz com que o sistema converse com o servidor "por baixo dos panos", mandando e recebendo os dados das vendas de forma super rápida e segura, deixando a navegação do vendedor muito mais leve.

4 DESENVOLVIMENTO DO PROJETO

O desenvolvimento do Adego Systems foi dividido em três partes principais: o Front-end (a tela que o usuário vê), o Back-end (o motor onde ficam as regras de negócio) e o Banco de Dados (onde salvamos as informações). Usamos uma adaptação do modelo arquitetural MVC (Model-View-Controller). Na prática, nosso servidor prepara a base das telas e o navegador do cliente busca as informações atualizadas de forma dinâmica, conversando com a nossa API.

4.1 Modelagem do Sistema

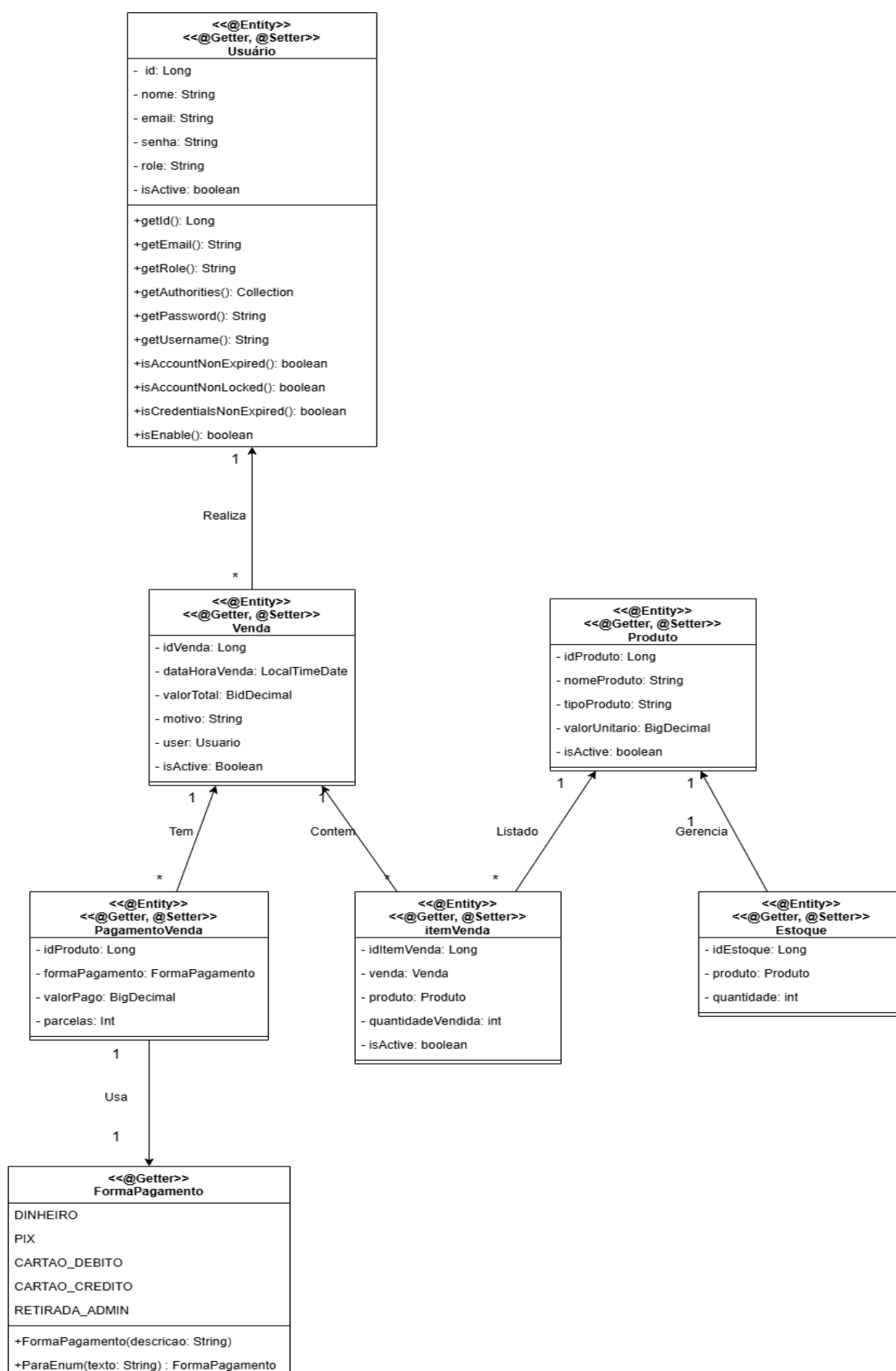
Antes de iniciarmos a programação, criamos mapas visuais para garantir que o sistema fosse bem estruturado. O primeiro deles é o Diagrama de Caso de Uso, que ilustra as interações dos usuários com o sistema, definindo o que o Vendedor e o Administrador podem realizar. O segundo é o Diagrama de Classes, que funciona como a "planta baixa" do nosso código Java, detalhando como as informações de produtos, vendas e usuários se conectam.

Figura 1: Diagrama de casos de uso



Fonte: Elaborado por Alef Hugo, Arthur Caselli, Richard Armond e Vitor Santana.

Figura 2: Diagrama de casos de uso



Fonte: Elaborado por Alef Hugo, Arthur Caselli, Richard Armond e Vitor Santana.

4.2 Front-end

A parte visual do sistema foi pensada para ser rápida, fácil de usar e com um visual bem profissional. Para montar as páginas de base, usamos a ferramenta Thymeleaf, que ajuda o servidor a entregar as telas de forma segura. O visual em si foi feito usando apenas CSS puro, com um tema escuro e dourado que descansa a vista durante o dia de trabalho. Aproveitamos o Flexbox e o Grid Layout do CSS para deixar o sistema totalmente responsivo, ou seja, ele se adapta sozinho se o gerente acessar pelo computador ou pelo celular, escondendo a barra lateral automaticamente quando a tela é pequena.

Para fazer o sistema funcionar de forma inteligente sem precisar recarregar a página toda hora, dividimos os arquivos JavaScript em pequenos módulos (como um arquivo só para vendas e outro só para relatórios). A comunicação com o servidor acontece nos bastidores usando a tecnologia Fetch API. Criamos uma função principal que pega o "crachá digital" do usuário (o token JWT) salvo no navegador e anexa automaticamente em todos os pedidos feitos ao sistema. Além disso, criamos alertas amigáveis: se o servidor avisar que uma senha está errada ou que faltou estoque, o nosso JavaScript entende o código do problema e mostra uma mensagem clara para o vendedor na tela.

Para ajudar na gestão da loja, incluímos um painel de indicadores (o Dashboard). Usamos uma biblioteca chamada Chart.js para transformar a lista de vendas diárias em gráficos de barras e anéis fáceis de ler. Assim, o dono da adega consegue bater o olho e ver rapidamente o faturamento da semana, como está a saída do estoque e qual vendedor está com o melhor desempenho.

4.3 Back-end

O motor principal do nosso sistema foi programado em Java (versão 21) usando o framework Spring Boot (versão 3.4.2). Organizamos o código em camadas separadas para que tudo fique bem arrumado e fácil de dar manutenção. A primeira porta de entrada são os controladores (Controllers). Eles recebem os pedidos da tela, conferem se os dados estão no formato correto e devolvem as respostas. Usamos regras de validação diretas no código — como impedir que o sistema aceite uma quantidade negativa de produtos ou um campo em branco — garantindo que nenhuma informação "suja" passe para a próxima etapa.

Depois que os dados entram e são validados, eles vão para a camada de serviços (Services), que é onde a verdadeira mágica do negócio acontece. O grande destaque aqui é o serviço de vendas. Ele atua como o maestro da loja: quando uma venda é fechada, ele não apenas salva o pedido, mas também vai ao estoque e dá a baixa automática dos produtos, além de registrar todas as formas de pagamento usadas pelo cliente. Como mexer com dinheiro e estoque é algo crítico, usamos a anotação `@Transactional`. Isso significa que, se a energia cair ou der algum erro bem no meio da operação, o sistema desfaz tudo automaticamente (rollback), evitando que o estoque seja descontado sem a venda ter sido realmente concluída.

Para manter o sistema sempre no ar e não assustar o usuário com aquelas telas cheias de códigos de erro do Java, criamos um controlador global de falhas. Se acontecer algum problema de operação, como tentar vender uma bebida que já acabou no estoque, essa classe "segura" o erro técnico e devolve apenas uma mensagem simples e organizada para a tela do caixa.

4.4 Banco de dados

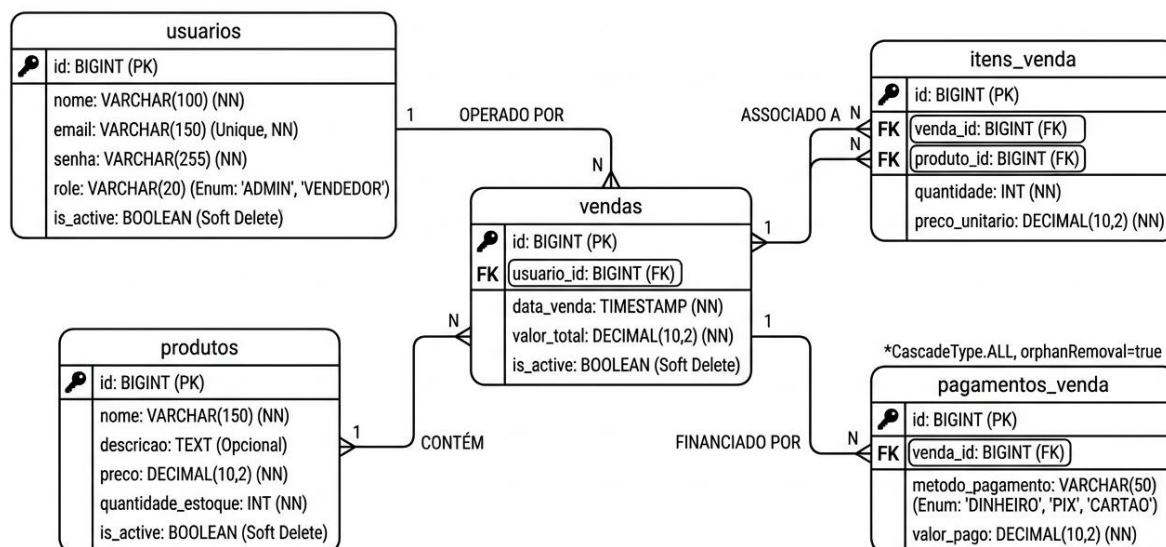
Para guardar todas as informações da loja de forma garantida, escolhemos o banco de dados MySQL e usamos a ferramenta Hibernate para facilitar a comunicação. Com ele, não precisamos escrever os complexos comandos SQL na mão; nós simplesmente dizemos ao código Java que uma determinada classe do sistema é uma tabela. Todas as nossas tabelas têm um código único de identificação gerado automaticamente pelo próprio banco, garantindo que nenhuma venda ou produto se misture.

As ligações entre as informações também são feitas de forma bem estruturada. Por exemplo, cada venda no sistema sabe exatamente qual vendedor a realizou e quais foram os pagamentos feitos nela. Configuramos o sistema para que, se uma venda for editada ou removida, os registros financeiros vinculados a ela sejam limpos automaticamente do banco, mantendo os dados sem sobras desnecessárias.

Uma regra muito importante que aplicamos foi a exclusão lógica (soft delete). No nosso sistema, nós não usamos o comando de apagar de verdade (DELETE). Se um administrador "excluir" um produto ou uma venda na tela, o sistema apenas marca aquele item como inativo no banco de dados. Isso é fundamental no

comércio, pois preserva o histórico de fechamentos de caixa e evita problemas nos relatórios do passado. Para facilitar a avaliação do projeto, também criamos uma rotina que já cadastra alguns produtos, um administrador de teste e algumas vendas prontas assim que o sistema liga pela primeira vez com o banco vazio.

Figura 3: estrutura lógica do banco de dados



Fonte: Elaborado por Alef Hugo, Arthur Caselli, Richard Armond e Vitor Santana.

4.5 Segurança da Informação

Sendo um dos pilares centrais do nosso projeto, criamos uma camada de proteção bastante moderna, que vai muito além de apenas exigir uma senha de acesso. O nosso sistema não usa o modelo antigo de guardar as sessões abertas na memória do servidor, o que gastaria muito processamento. Nós adotamos o modelo stateless com JSON Web Tokens (JWT). Funciona da seguinte maneira: quando o usuário entra no sistema, ele recebe um "crachá digital" validado e assinado com criptografia forte. Para garantir que nenhum programa malicioso roube esse passe facilmente, nós o trafegamos de forma rigorosa, incluindo a utilização de Cookies com restrição de site, prevenindo ataques de redirecionamento falsificado (CSRF).

Além de proteger a porta de entrada, nós protegemos os dados que ficam guardados. O sistema nunca salva a senha do usuário pura no banco de dados. Nós aplicamos a tecnologia BCrypt, que transforma a senha original num código totalmente embaralhado e irreversível. Assim, mesmo que alguém conseguisse

invadir fisicamente o servidor do banco de dados, seria impossível descobrir as senhas dos funcionários.

Para fechar a segurança, implementamos o Controle de Acesso Baseado em Perfis (RBAC), que atua como um segurança duplo. Na interface visual, o nosso código analisa o tipo de funcionário logado e simplesmente esconde botões e páginas que ele não tem permissão para ver (como a tela de exclusão de vendas). Mas não confiamos apenas na tela visual. Lá no back-end, usamos bloqueios diretos no código Java. Se um vendedor tentar usar uma ferramenta externa de hacker para forçar o cancelamento de uma venda (que é uma ação exclusiva do gestor), a nossa camada de segurança barra o pedido imediatamente, garantindo que cada pessoa só consiga fazer exatamente aquilo que a sua função permite.

5 TESTES E RESULTADOS

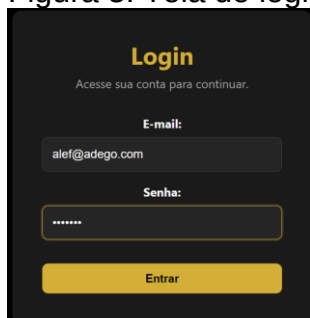
Para garantir que o Adego Systems estivesse pronto para o uso real e livre de falhas que pudessem prejudicar o fluxo de um pequeno negócio, o sistema passou por uma bateria de testes funcionais. O objetivo dessa etapa foi validar na prática se todas as regras que programamos no *back-end* refletiam corretamente na interface visual do *front-end*, garantindo a estabilidade e a segurança das operações.

Durante os testes, simulamos o dia a dia de uma adega: acessamos o sistema com diferentes perfis de usuário para verificar o bloqueio de telas (testando a eficácia do JWT e do RBAC), realizamos cadastros de novos produtos para validar a comunicação com o banco de dados e executamos o fluxo completo de um caixa, incluindo a divisão de pagamentos em múltiplas formas (dinheiro, PIX e cartão). Também forçamos situações de erro — como tentar vender um item com estoque zerado ou usar uma senha incorreta — para certificar que o tratamento de exceções exibiria os alertas amigáveis na tela, sem travar o sistema.

Para além dos testes de fluxo do caixa, precisávamos garantir que a plataforma suportaria o acúmulo de informações ao longo do tempo. Para isso, utilizamos Inteligência Artificial para gerar um grande volume de dados fictícios e estruturados. Com o auxílio dessa IA, inserimos no banco de dados com mais de 980 registros de vendas simuladas. Essa volumetria de massa de dados foi essencial para testar o desempenho do sistema sob carga e validar se o processamento lógico do painel de indicadores (*Dashboard*) — que compila as métricas financeiras diárias e mensais — continuaria rápido e preciso mesmo lidando com centenas de cálculos e renderizações gráficas simultâneas.

Abaixo, apresentamos as capturas de tela das principais funcionalidades validadas durante esses testes, comprovando o êxito e o funcionamento integrado de toda a arquitetura desenvolvida.

Figura 3: Tela de login



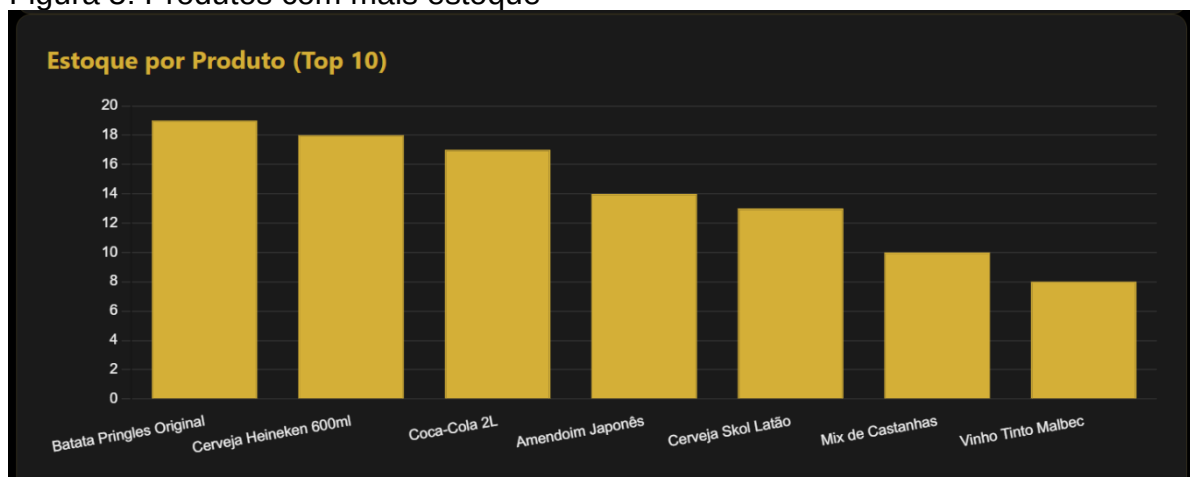
Fonte: Elaborado por Alef Hugo, Arthur Caselli, Richard Armond e Vitor Santana.

Figura 4: SlideBar / Início



Fonte: Elaborado por Alef Hugo, Arthur Caselli, Richard Armond e Vitor Santana.

Figura 5: Produtos com mais estoque



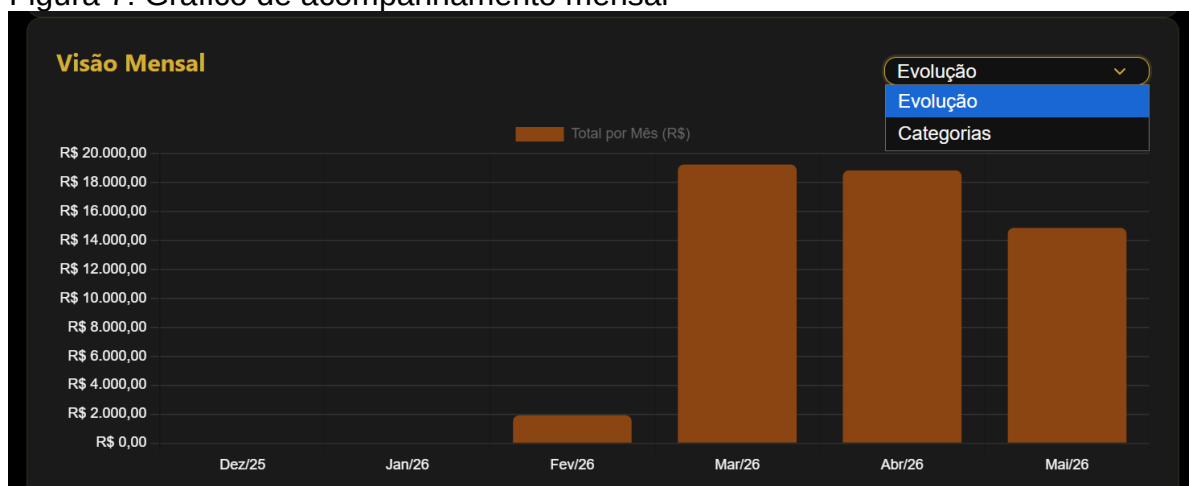
Fonte: Elaborado por Alef Hugo, Arthur Caselli, Richard Armond e Vitor Santana.

Figura 6: Produtos com mais estoque / Últimas vendas



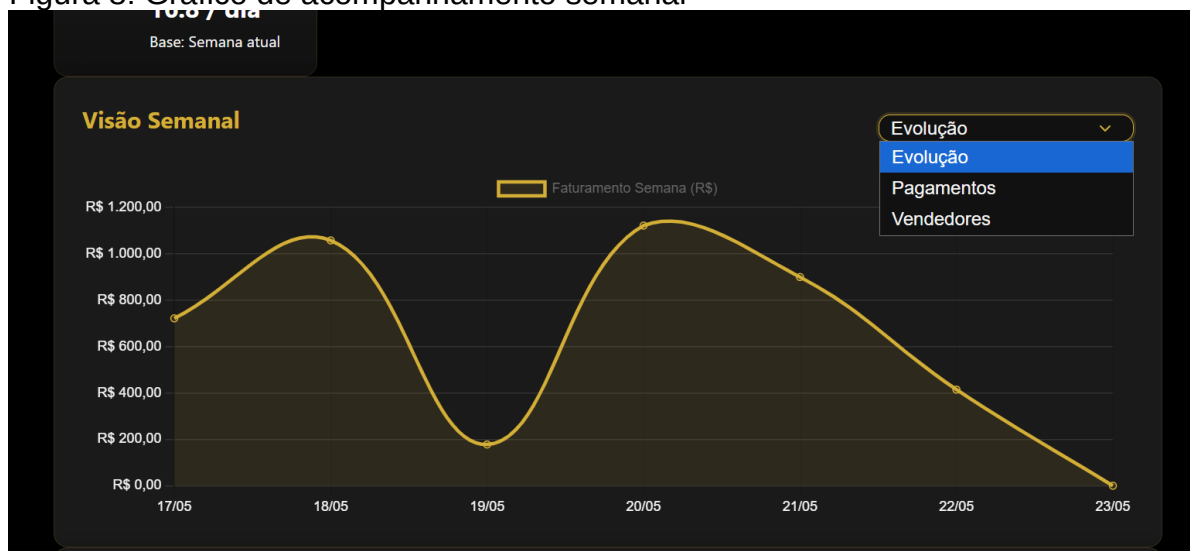
Fonte: Elaborado por Alef Hugo, Arthur Caselli, Richard Armond e Vitor Santana.

Figura 7: Gráfico de acompanhamento mensal



Fonte: Elaborado por Alef Hugo, Arthur Caselli, Richard Armond e Vitor Santana.

Figura 8: Gráfico de acompanhamento semanal



Fonte: Elaborado por Alef Hugo, Arthur Caselli, Richard Armond e Vitor Santana.

Figura 9: Cadastro de usuário (apenas admin)

Usuários - Adego Systems

localhost:8080/usuarios

Novo Usuário

Nome:

E-mail:

Senha:

Cancelar Salvar

Fonte: Elaborado por Alef Hugo, Arthur Caselli, Richard Armond e Vitor Santana.

Figura 10: Tabela de usuário

The screenshot shows a web application interface for 'Adego Systems' with a sidebar menu and a main content area titled 'Gestão de Acessos'. The sidebar menu includes options like Dashboard, Produtos, Estoque, Vendas, Usuários (highlighted), and Sair. The main content area has a '+ Novo Usuário' button and a table of users.

NOME	E-MAIL	PERFIL	STATUS	AÇÕES
Administrador	admin@admin.com	Admin	Ativo	-
Alef	alef@adeago.com	Vendedor	Ativo	Desativar
Richard	richard@adeago.com	Vendedor	Ativo	Desativar
Vitor	vitor@adeago.com	Vendedor	Ativo	Desativar
Arthur	arthur@adeago.com	Vendedor	Ativo	Desativar

Fonte: Elaborado por Alef Hugo, Arthur Caselli, Richard Armond e Vitor Santana.

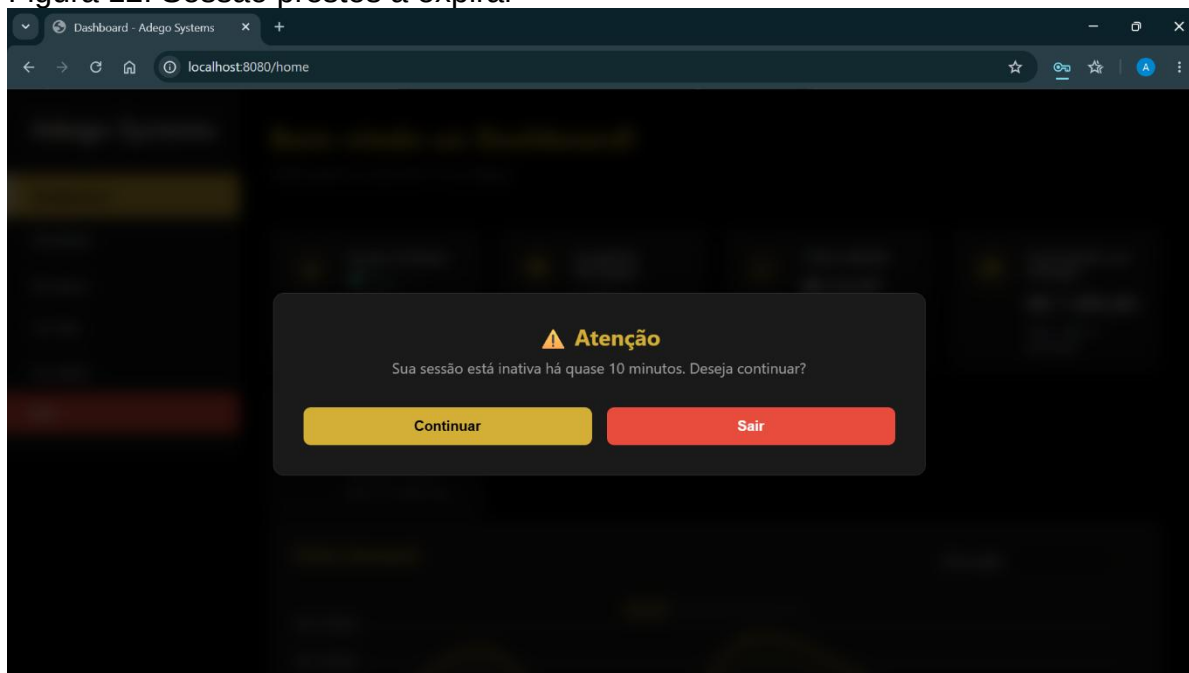
Figura 11: Confirmação de desativação de usuário

The screenshot shows the same web application interface as Figure 10, but with a confirmation dialog box overlaid. The dialog box is titled 'localhost:8080 diz' and contains the text 'Deseja realmente desativar este usuário?'. It has two buttons: 'OK' and 'Cancelar'.

NOME	E-MAIL	PERFIL	STATUS	AÇÕES
Administrador	admin@admin.com	Admin	Ativo	-
Alef	alef@adeago.com	Vendedor	Ativo	Desativar
Richard	richard@adeago.com	Vendedor	Ativo	Desativar
Vitor	vitor@adeago.com	Vendedor	Ativo	Desativar
Arthur	arthur@adeago.com	Vendedor	Ativo	Desativar

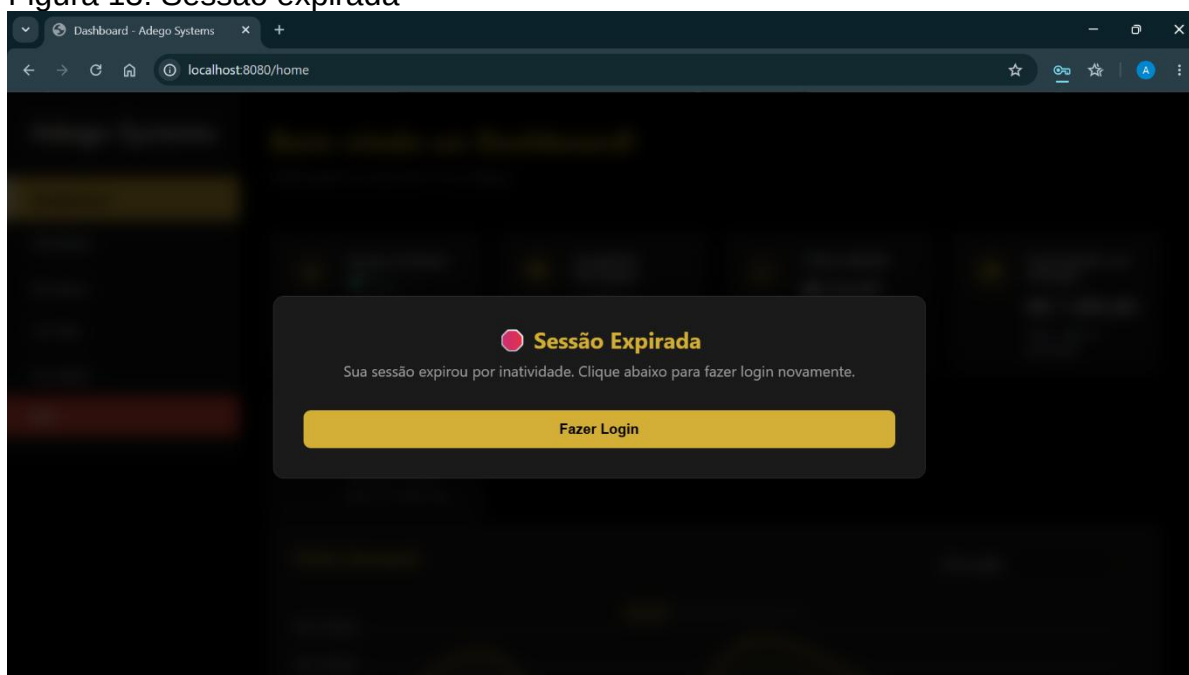
Fonte: Elaborado por Alef Hugo, Arthur Caselli, Richard Armond e Vitor Santana.

Figura 12: Sessão prestes a expirar



Fonte: Elaborado por Alef Hugo, Arthur Caselli, Richard Armond e Vitor Santana.

Figura 13: Sessão expirada



Fonte: Elaborado por Alef Hugo, Arthur Caselli, Richard Armond e Vitor Santana.

Figura 14: Tabela de estoque (admin)

PRODUTO	CATEGORIA	QUANTIDADE	STATUS	AÇÕES
Vinho Tinto Malbec	Vinho	0	Crítico	Atualizar Baixa
Cerveja Heineken 600ml	Cerveja	8	Baixo	Atualizar Baixa
Cerveja Skol Latão	Cerveja	13	OK	Atualizar Baixa
Coca-Cola 2L	Refrigerante	17	OK	Atualizar Baixa
Batata Pringles Original	Snack	19	OK	Atualizar Baixa
Amendoim Japonês	Snack	14	OK	Atualizar Baixa

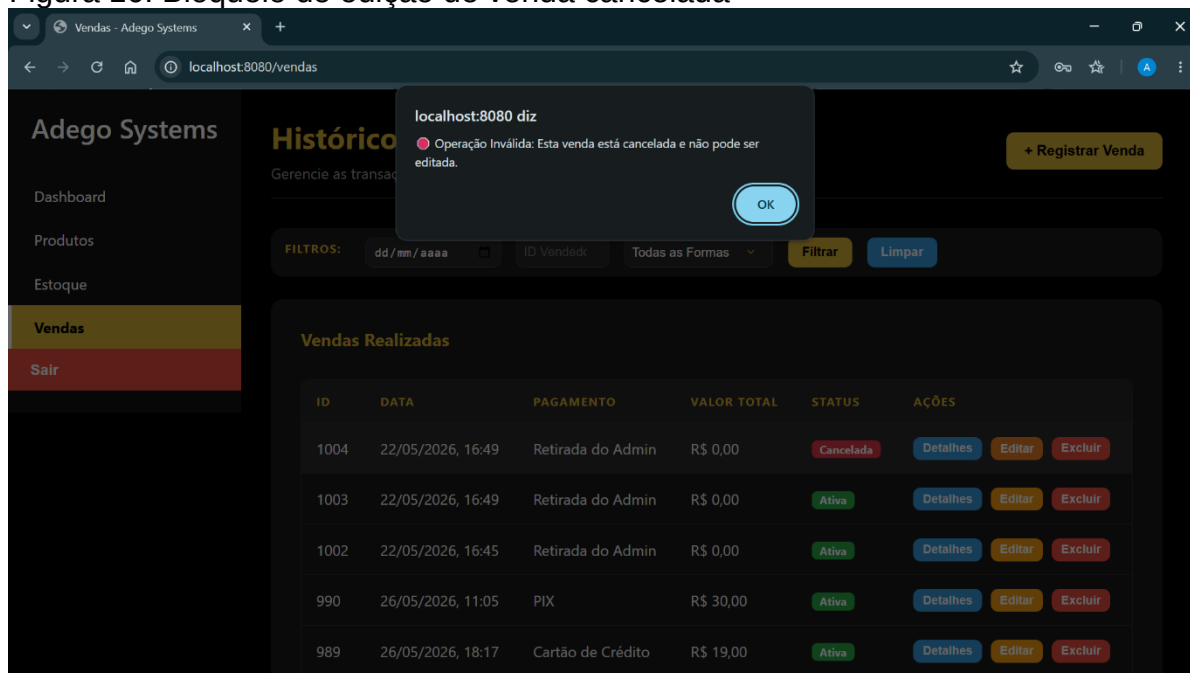
Fonte: Elaborado por Alef Hugo, Arthur Caselli, Richard Armond e Vitor Santana.

Figura 15: Exclusão de venda (admin)

ID	DATA	PAGAMENTO	VALOR TOTAL	STATUS
1004	22/05/2026, 16:49	Retirada do Admin	R\$ 0,00	Ativa
1003	22/05/2026, 16:49	Retirada do Admin	R\$ 0,00	Ativa
1002	22/05/2026, 16:45	Retirada do Admin	R\$ 0,00	Ativa
990	26/05/2026, 11:05	PIX	R\$ 30,00	Ativa
989	26/05/2026, 18:17	Cartão de Crédito	R\$ 19,00	Ativa
988	23/05/2026, 19:31	Cartão de Débito	R\$ 12,00	Ativa

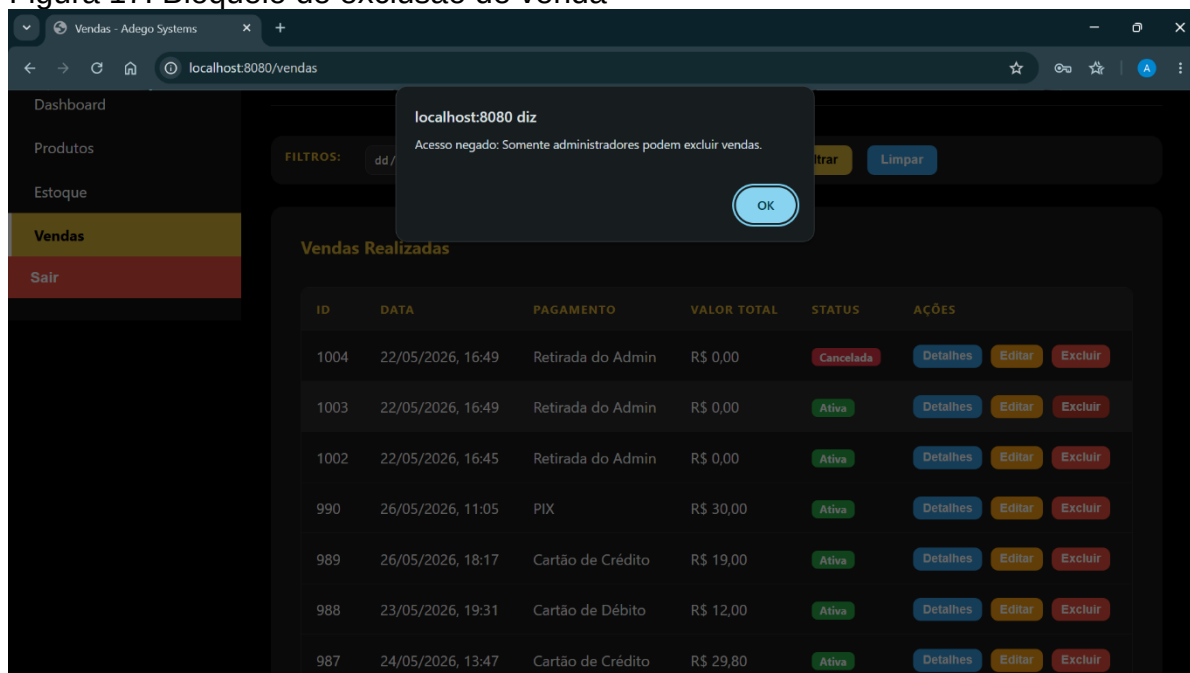
Fonte: Elaborado por Alef Hugo, Arthur Caselli, Richard Armond e Vitor Santana.

Figura 16: Bloqueio de edição de venda cancelada



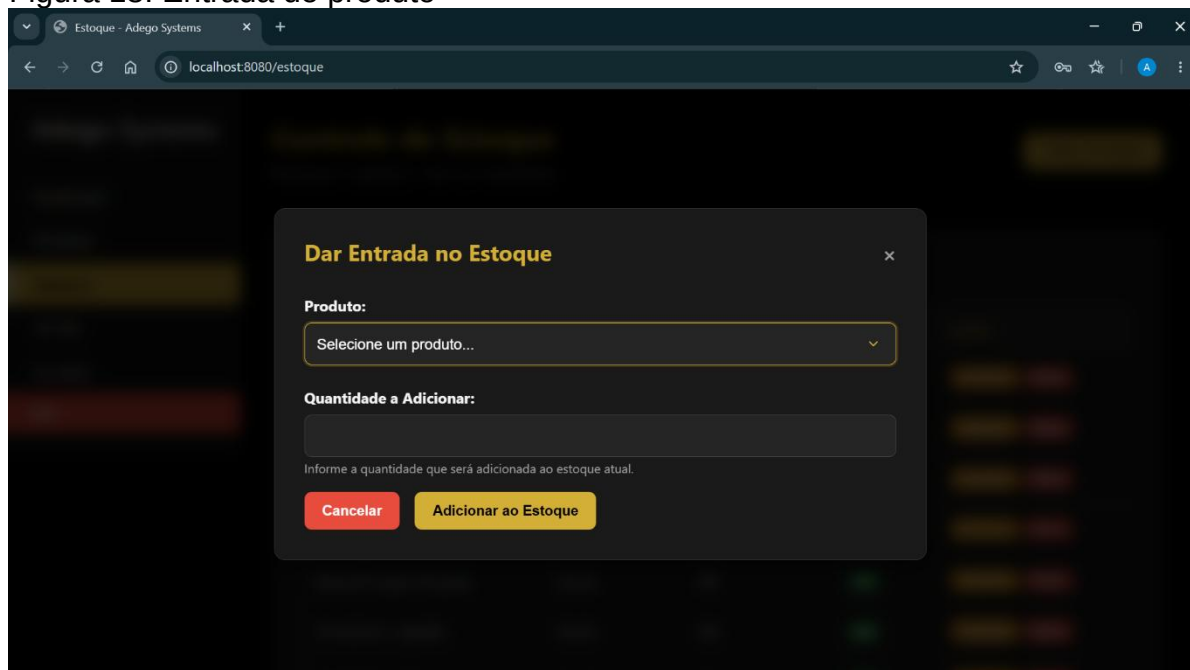
Fonte: Elaborado por Alef Hugo, Arthur Caselli, Richard Armond e Vitor Santana.

Figura 17: Bloqueio de exclusão de venda



Fonte: Elaborado por Alef Hugo, Arthur Caselli, Richard Armond e Vitor Santana.

Figura 18: Entrada de produto



Dar Entrada no Estoque

Produto:

Selecione um produto...

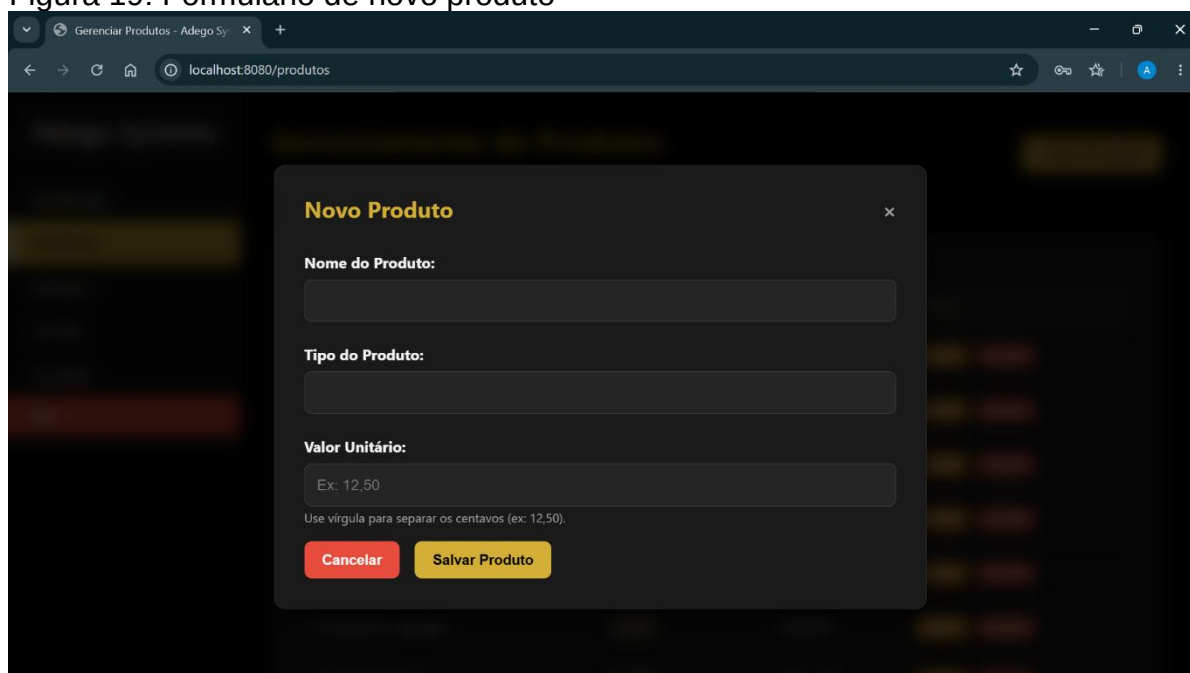
Quantidade a Adicionar:

Informe a quantidade que será adicionada ao estoque atual.

Cancelar **Adicionar ao Estoque**

Fonte: Elaborado por Alef Hugo, Arthur Caselli, Richard Armond e Vitor Santana.

Figura 19: Formulário de novo produto



Novo Produto

Nome do Produto:

Tipo do Produto:

Valor Unitário:

Ex: 12,50

Use vírgula para separar os centavos (ex: 12,50).

Cancelar **Salvar Produto**

Fonte: Elaborado por Alef Hugo, Arthur Caselli, Richard Armond e Vitor Santana.

Figura 20: Produtos quando admin

Adego Systems

Dashboard

Produtos

Estoque

Vendas

Usuários

Sair

Gerenciamento de Produtos

Adicione, edite ou remova produtos do seu catálogo.

+ Novo Produto

NOME	TIPO	VALOR	AÇÕES
Vinho Tinto Malbec	Vinho	R\$ 85,90	Editar Excluir
Cerveja Heineken 600ml	Cerveja	R\$ 9,50	Editar Excluir
Cerveja Skol Latão	Cerveja	R\$ 5,50	Editar Excluir
Coca-Cola 2L	Refrigerante	R\$ 12,00	Editar Excluir
Batata Pringles Original	Snack	R\$ 14,90	Editar Excluir
Amendoim Japonês	Snack	R\$ 8,50	Editar Excluir

Fonte: Elaborado por Alef Hugo, Arthur Caselli, Richard Armond e Vitor Santana.

Figura 21: Produtos quando vendedor

Adego Systems

Dashboard

Produtos

Estoque

Vendas

Sair

Controle de Estoque

Monitore e reponha o nível de mercadorias.

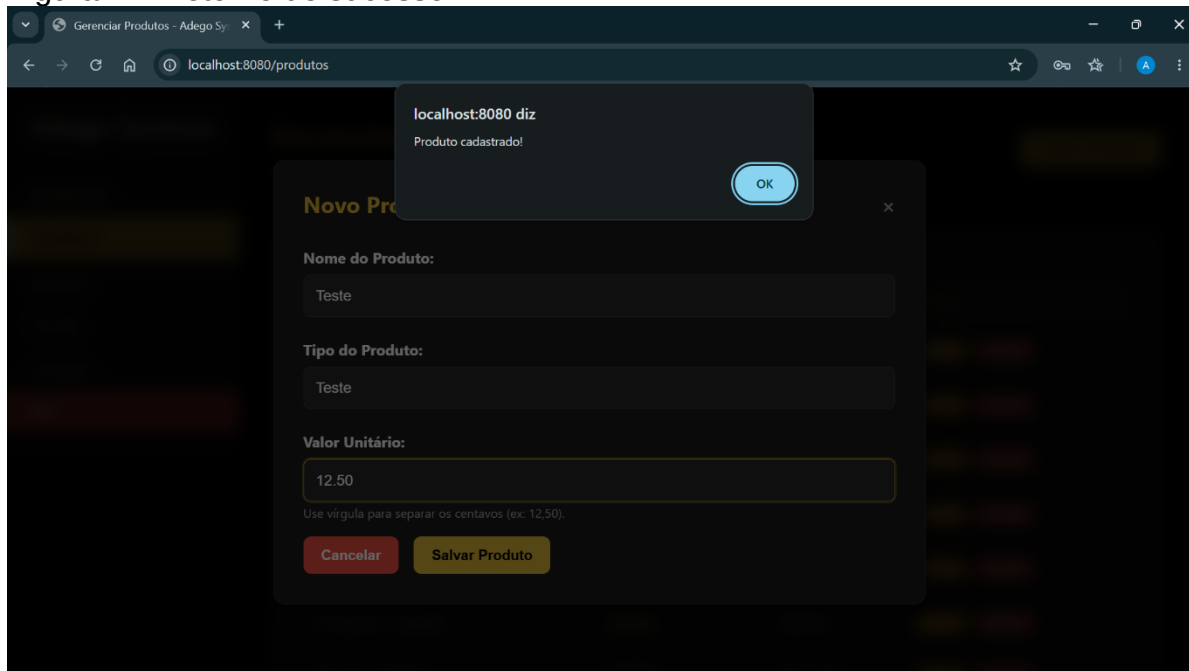
+ Nova Entrada

Níveis de Estoque

PRODUTO	CATEGORIA	QUANTIDADE	STATUS	AÇÕES
Vinho Tinto Malbec	Vinho	0	Crítico	Atualizar
Cerveja Heineken 600ml	Cerveja	8	Baixo	Atualizar
Cerveja Skol Latão	Cerveja	13	OK	Atualizar
Coca-Cola 2L	Refrigerante	17	OK	Atualizar
Batata Pringles Original	Snack	19	OK	Atualizar
Amendoim Japonês	Snack	14	OK	Atualizar

Fonte: Elaborado por Alef Hugo, Arthur Caselli, Richard Armond e Vitor Santana.

Figura 22: Retorno de sucesso



Fonte: Elaborado por Alef Hugo, Arthur Caselli, Richard Armond e Vitor Santana.

Figura 23: Página de vendas

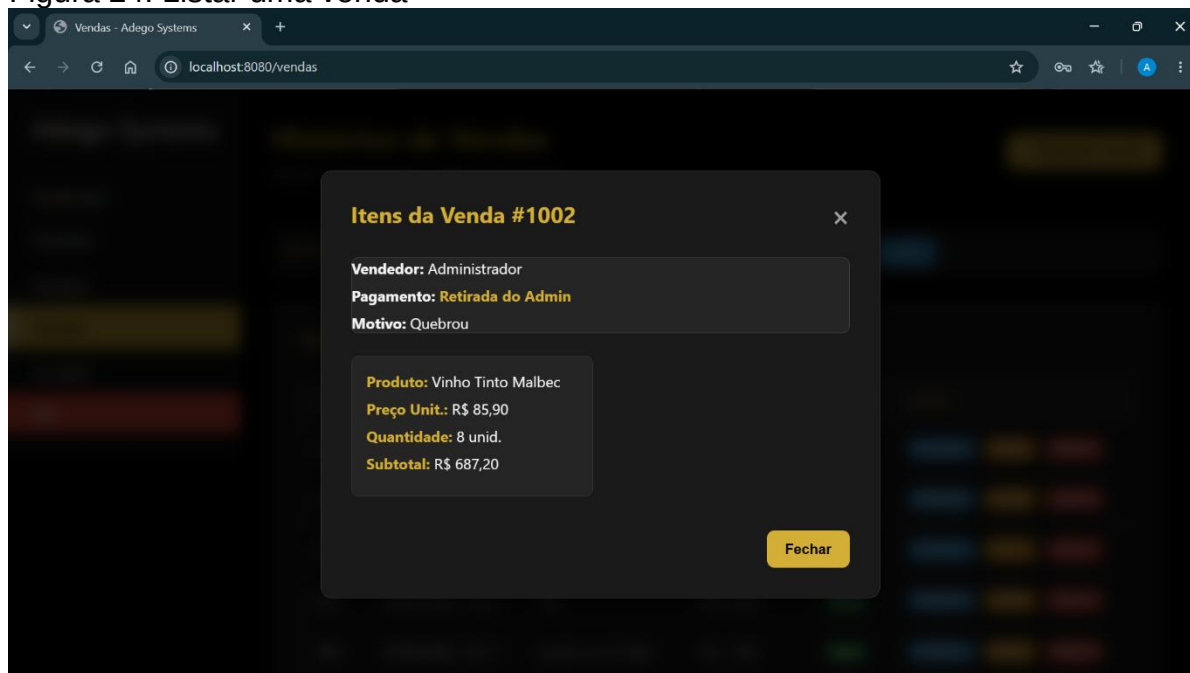
A imagem mostra uma interface web no navegador com o endereço `localhost:8080/vendas`. A página exibe uma tabela de vendas com 10 itens por página. A tabela possui as seguintes colunas: ID, Data e Hora, Descrição, Valor, Status e Ações. O status "Ativa" é indicado por um ícone verde, e "Cancelada" por um ícone vermelho. As ações disponíveis para cada item são "Detalhes", "Editar" e "Excluir".

ID	Data e Hora	Descrição	Valor	Status	Ações
1004	22/05/2026, 16:49	Retirada do Admin	R\$ 0,00	Cancelada	Detalhes Editar Excluir
1003	22/05/2026, 16:49	Retirada do Admin	R\$ 0,00	Ativa	Detalhes Editar Excluir
1002	22/05/2026, 16:45	Retirada do Admin	R\$ 0,00	Ativa	Detalhes Editar Excluir
990	26/05/2026, 11:05	PIX	R\$ 30,00	Ativa	Detalhes Editar Excluir
989	26/05/2026, 18:17	Cartão de Crédito	R\$ 19,00	Ativa	Detalhes Editar Excluir
988	23/05/2026, 19:31	Cartão de Débito	R\$ 12,00	Ativa	Detalhes Editar Excluir
987	24/05/2026, 13:47	Cartão de Crédito	R\$ 29,80	Ativa	Detalhes Editar Excluir
986	27/05/2026, 01:40	Cartão de Crédito	R\$ 22,00	Ativa	Detalhes Editar Excluir
985	24/05/2026, 18:24	Cartão de Crédito	R\$ 24,00	Ativa	Detalhes Editar Excluir
984	23/05/2026, 07:23	Cartão de Crédito	R\$ 9,50	Ativa	Detalhes Editar Excluir

Na base da tabela, há botões "Anterior" e "Próxima". Abaixo da tabela, há uma barra de paginação que indica "Página 1 de 100 (993 vendas)" e um seletor de itens por página com o valor "10 itens".

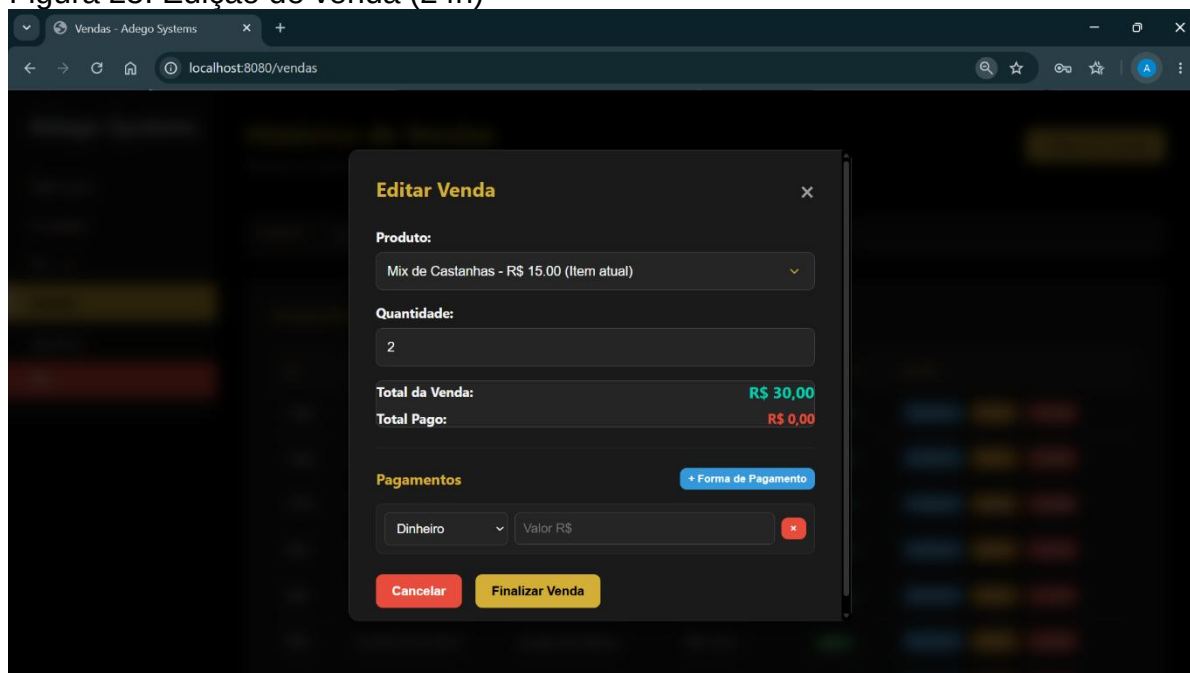
Fonte: Elaborado por Alef Hugo, Arthur Caselli, Richard Armond e Vitor Santana.

Figura 24: Listar uma venda



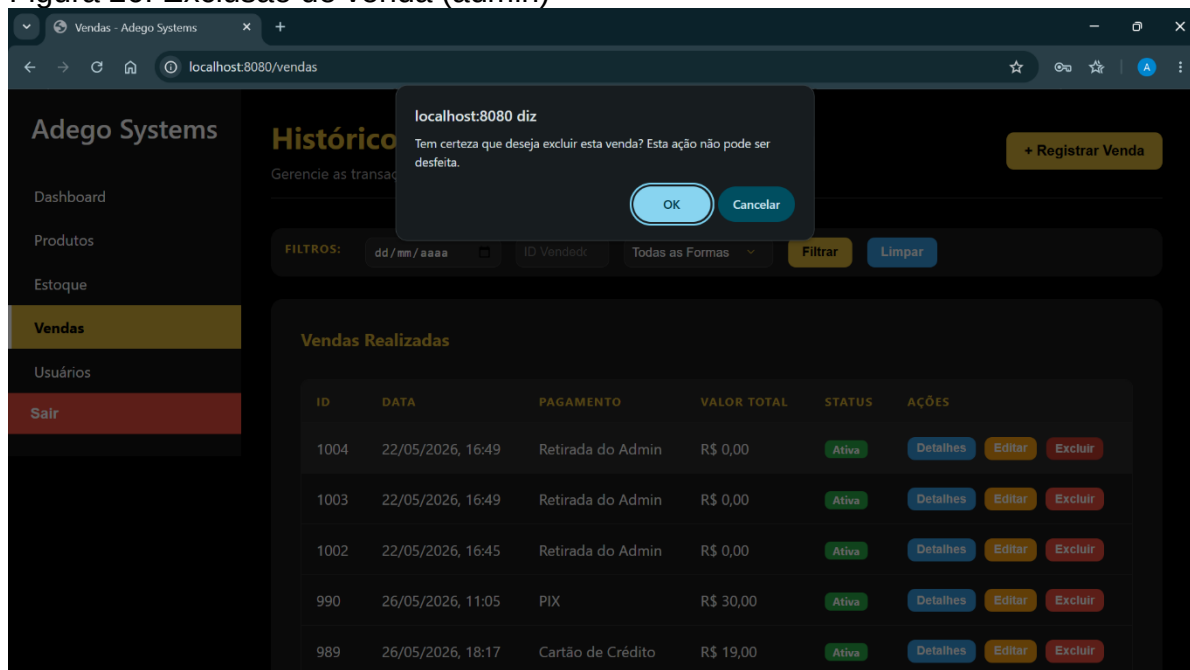
Fonte: Elaborado por Alef Hugo, Arthur Caselli, Richard Armond e Vitor Santana

Figura 25: Edição de venda (24h)



Fonte: Elaborado por Alef Hugo, Arthur Caselli, Richard Armond e Vitor Santana

Figura 26: Exclusão de venda (admin)



Fonte: Elaborado por Alef Hugo, Arthur Caselli, Richard Armond e Vitor Santana

Figura 27: Confirmação de exclusão de venda (admin)



Fonte: Elaborado por Alef Hugo, Arthur Caselli, Richard Armond e Vitor Santana

6 CONCLUSÃO

O desenvolvimento do Adego Systems provou na prática que é possível criar uma ferramenta simples de usar, mas muito segura, para automatizar pequenos negócios. Usando as tecnologias Java, Spring Boot e MySQL, conseguimos entregar um sistema que tira o peso das operações manuais e protege as informações da loja. Cumprimos os objetivos que definimos no início do projeto: a tela do caixa ficou rápida e fácil de mexer, permitindo que o gerente acompanhe o lucro pelas métricas dos gráficos interativos (dashboards). Além disso, a dor de cabeça com o controle manual foi resolvida, já que o estoque agora é descontado automaticamente a cada venda ou retirada, acabando de vez com as falhas humanas no papel.

Durante a programação, o nosso maior desafio técnico foi fazer a segurança funcionar perfeitamente em conjunto com as requisições da tela visual. Configurar o sistema para não guardar sessões na memória (stateless) e usar os "crachás digitais" (JSON Web Tokens - JWT) exigiu muito estudo da nossa parte. Precisávamos garantir que o token fosse criado, guardado e enviado em segurança pelo JavaScript a cada clique do usuário. Outro grande obstáculo foi dominar o controle do banco de dados na hora de salvar vendas complexas (aquelas com muitos produtos e pagamentos divididos em dinheiro, cartão e PIX). Tivemos que aprender a usar recursos do Java, como a anotação `@Transactional`, para garantir que, se algo desse erro no meio da transação, o sistema desfizesse a operação inteira sozinho, evitando que o estoque fosse descontado sem o dinheiro realmente entrar no caixa.

Apesar dessas dificuldades, construir o sistema nos trouxe um aprendizado prático gigantesco sobre Engenharia de Software e, de forma muito clara, sobre Segurança da Informação. Entendemos o real valor de criar um sistema pensando na segurança desde o primeiro rascunho (Security by Design). Vimos funcionar na prática ferramentas essenciais, como a criptografia irreversível de senhas com o BCrypt, o bloqueio de funções dependendo do cargo do funcionário (RBAC) e o cuidado de não vaziar dados sensíveis usando o padrão DTO. Como próximos passos para o futuro do software, recomendamos integrar uma API para aprovar pagamentos online direto no sistema e, quem sabe, expandir o projeto para um aplicativo de celular, deixando a gestão da adega literalmente na palma da mão do dono, 24 horas por dia.

7 REFERÊNCIAS

- AUTH0. Java JWT: JSON Web Token for Java. Disponível em: <https://github.com/auth0/java-jwt>. Acesso em: 02 maio 2026.
- CHART.JS. Chart.js Documentation. Disponível em: <https://www.chartjs.org/docs/latest/>. Acesso em: 02 maio 2026.
- MDN WEB DOCS. JavaScript Guide. Disponível em: <https://developer.mozilla.org/>. Acesso em: 02 maio 2026.
- ORACLE. Java 21 Documentation. Disponível em: <https://docs.oracle.com/en/java/javase/21/>. Acesso em: 02 maio 2026.
- ORACLE. MySQL 8.0 Reference Manual. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/>. Acesso em: 02 maio 2026.
- SPRING. Spring Boot Reference Guide. Disponível em: <https://docs.spring.io/spring-boot/docs/current/reference/html/>. Acesso em: 02 maio 2026.
- SPRING. Spring Security Reference. Disponível em: <https://docs.spring.io/spring-security/reference/index.html>. Acesso em: 02 maio 2026.
- THYMELEAF. Thymeleaf Documentation. Disponível em: <https://www.thymeleaf.org/documentation.html>. Acesso em: 16 maio 2026.
- W3C. HTML5 Specification. Disponível em: <https://www.w3.org/TR/html5/>. Acesso em: 16 maio 2026.